PENJELASAN PROYEK

Asisten — AI Chatbot

Chatbot web yang jawabannya mengalir kata per kata dan bisa dihentikan kapan saja, dengan backend Go yang bisa memakai Gemini, OpenAI, atau Anthropic.

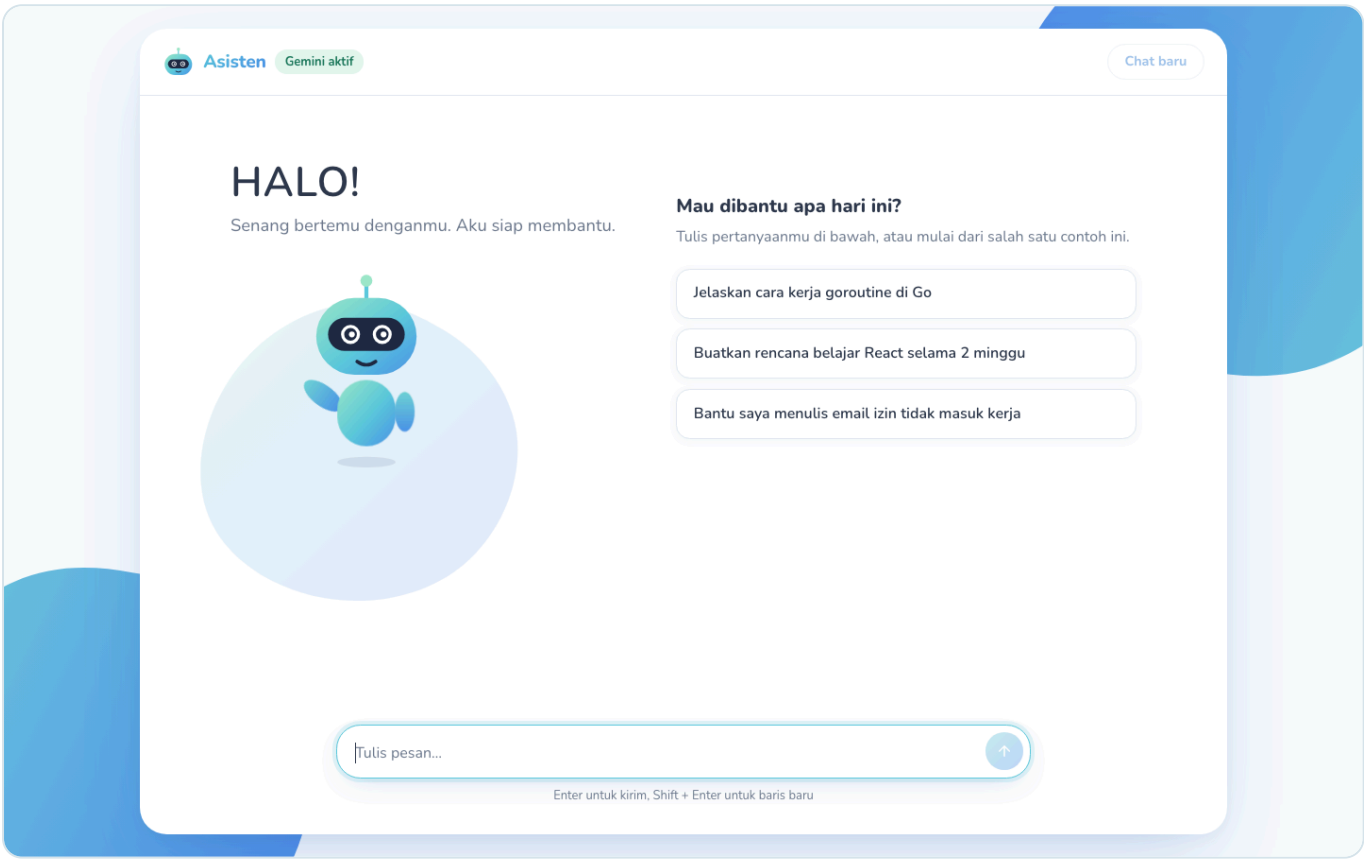
Raihan Evanza · Fullstack & Mobile Developer · portfolio-raihan-evanza.vercel.app

PERAN
Proyek pribadi · backend Go, frontend React, desain UI

PLATFORM
Web (desktop & ponsel), dijalankan lokal

TAUTAN
Kode sumber (GitHub): github.com/raihanevanza/chat-bot-go

Go 1.22 (standard library) · React 18 · Vite 5 · Server-Sent Events · Gemini API · OpenAI API · Anthropic API



RINGKASAN

Chatbot yang menunggu seluruh jawaban selesai sebelum menampilkannya terasa lambat, karena jawaban model bahasa bisa butuh beberapa detik. Proyek ini saya buat untuk mempelajari cara menampilkan jawaban secara bertahap, sekaligus belajar menulis backend dengan Go.

Hasilnya adalah aplikasi chat dengan backend Go yang hanya memakai standard library dan frontend React + Vite. Backend meneruskan potongan jawaban dari provider AI ke browser lewat Server-Sent Events, dan pengguna bisa menghentikan jawaban di tengah jalan. Provider dipilih otomatis dari API key yang tersedia (Gemini, OpenAI, atau Anthropic).

Tanpa API key sama sekali, backend masuk mode demo yang tetap men-stream jawaban contoh, sehingga siapa pun bisa meng-clone repo dan mencoba alurnya dalam beberapa menit tanpa biaya.

FITUR UTAMA

Jawaban mengalir real-time
Setiap potongan teks dari model langsung dikirim ke browser sebagai event SSE dan ditambahkan ke gelembung chat, jadi kata pertama muncul tanpa menunggu jawaban selesai.

Hentikan jawaban di tengah jalan
Selama jawaban berjalan, tombol kirim berubah menjadi tombol stop. Bagian jawaban yang sudah tampil tetap disimpan di percakapan.

Tiga provider AI
Gemini, OpenAI, dan Anthropic diimplementasikan di balik satu interface Provider di Go. Backend memilih yang pertama punya API key, dengan urutan Gemini, OpenAI, lalu Anthropic.

Mode demo tanpa API key
Jika semua API key kosong, backend memakai provider tiruan yang men-stream jawaban contoh kata per kata (sekitar 35 ms per kata) dan bisa dihentikan seperti jawaban sungguhan.

Badge status backend
Saat halaman dibuka, frontend memanggil /api/health dan menampilkan status di header: "Gemini aktif", "Terhubung", "Mode demo", atau "Backend mati".

Pesan error yang bisa dipahami
Error dari API provider diterjemahkan menjadi pesan untuk pengguna, misalnya kuota Gemini habis, kredit OpenAI habis, atau API key ditolak, lalu ditampilkan di atas kolom input.

Contoh pertanyaan & chat baru
Layar awal menampilkan tiga contoh pertanyaan yang bisa langsung diklik. Tombol "Chat baru" menghentikan jawaban yang sedang berjalan dan mengosongkan percakapan.

Tampilan ramah & responsif
Desain cerah dengan maskot robot, indikator mengetik sebelum kata pertama tiba, gulir otomatis ke pesan terbaru, dan kolom input yang membesar mengikuti isi. Enter untuk kirim, Shift + Enter untuk baris baru.

ARSITEKTUR & ALUR DATA

Browser (React)
Menyimpan riwayat chat di state, mengirim POST /api/chat, dan membaca stream dengan fetch + ReadableStream.

→

Vite proxy
Saat development, request /api diteruskan ke localhost:8080 sehingga tidak perlu CORS.

→

Go ChatHandler
Memvalidasi dan memotong riwayat, lalu menulis event SSE dan melakukan flush setiap potongan.

→

Provider
Gemini, OpenAI, Anthropic, atau Mock; dipilih sekali saat server start.

→

API model AI
Endpoint streaming milik provider; tidak dipanggil dalam mode demo.

- Backend stateless: setiap pesan baru mengirim ulang seluruh riwayat, lalu backend hanya meneruskan 40 pesan terakhir (MAX_HISTORY) dan memastikan percakapan diawali pesan user.
- Format event ke browser sama untuk semua provider: {"type":"delta"}, {"type":"done"}, atau {"type":"error"}. Frontend tidak perlu tahu provider mana yang dipakai.
- Setiap provider membaca stream SSE dari API-nya sendiri dengan bufio.Scanner, mengambil teksnya, lalu memanggil callback onDelta yang menulis event ke browser.
- Untuk produksi, backend dibangun jadi satu file binary dan frontend jadi file statis; alamat backend diatur lewat VITE_API_URL dan ALLOWED_ORIGIN.

TEKNOLOGI PER LAPISAN

Backend	Go 1.22 · net/http ServeMux · encoding/json · bufio · context
Frontend	React 18 · Vite 5 · Fetch API + ReadableStream · AbortController · Plain CSS
AI	Gemini API (streamGenerateContent) · OpenAI API (Chat Completions) · Anthropic API (Messages) · Mock provider
Pengujian & build	go test (httptest) · go build · vite build

KEPUTUSAN TEKNIS

1. **SSE, bukan WebSocket**
   Data hanya mengalir satu arah, dari server ke browser, selama satu jawaban. SSE cukup memakai respons HTTP biasa (text/event-stream + Flush), tanpa protokol tambahan atau koneksi yang harus dijaga. Frontend membacanya dengan fetch, bukan EventSource, karena EventSource hanya mendukung GET dan tidak bisa mengirim riwayat chat di body.

2. **Stop memutus rantai sampai ke provider**
   Tombol stop memanggil AbortController.abort(), yang memutus request fetch. Di Go, r.Context() ikut dibatalkan, dan karena request ke API provider dibuat dengan NewRequestWithContext, request itu juga terputus sehingga model berhenti menghasilkan token. Handler lalu keluar tanpa mengirim pesan error, dan frontend mengabaikan AbortError supaya jawaban sebagian tetap tampil.

3. **Provider dipilih dari API key**
   Daripada menambah variabel konfigurasi "provider", backend cukup melihat key mana yang terisi. Gemini diprioritaskan karena key gratisnya mudah dibuat di Google AI Studio. Menambah provider baru cukup dengan satu file yang mengimplementasikan Name() dan Stream().

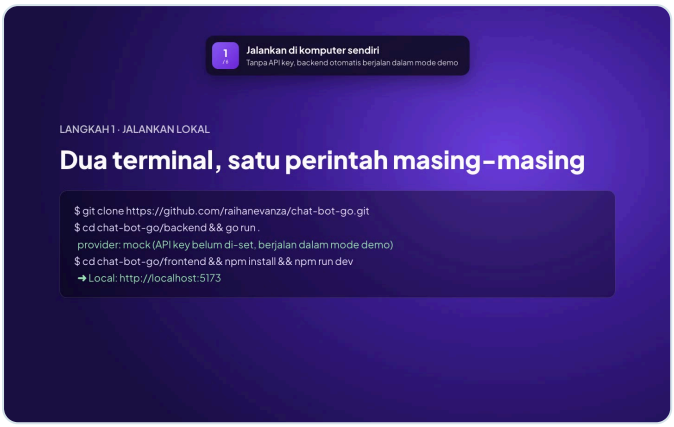
4. **Tanpa dependency eksternal di Go**
   go.mod tidak punya satu pun require. Routing memakai ServeMux Go 1.22 yang sudah mendukung pola "POST /api/chat", pemanggilan API provider memakai net/http langsung, dan file .env dibaca oleh parser kecil buatan sendiri yang tidak menimpa environment variable yang sudah ada. Tujuannya agar saya memahami tiap lapisan dan agar build cukup "go build".

5. **Mode demo sebagai fallback**
   Repo publik sebaiknya bisa dicoba tanpa mendaftar ke layanan berbayar. Provider mock memakai interface yang sama dan menghormati pembatalan context, jadi streaming, tombol stop, dan badge status bisa diuji persis seperti dengan model sungguhan.

6. **Streaming yang tidak tertahan proxy**
   Handler mengirim header Cache-Control: no-cache dan X-Accel-Buffering: no, lalu memanggil Flush setelah setiap event agar nginx atau proxy lain tidak menumpuk respons. WriteTimeout server sengaja tidak di-set karena jawaban panjang bisa berjalan lama.

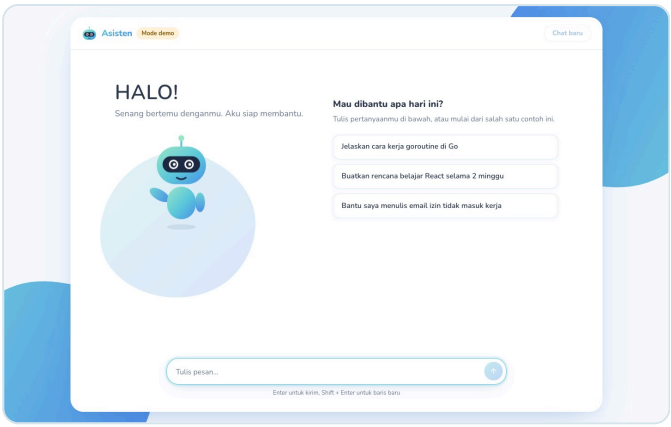
PANDUAN PENGGUNAAN

Langkah-langkah ini sama dengan urutan di video tutorial.



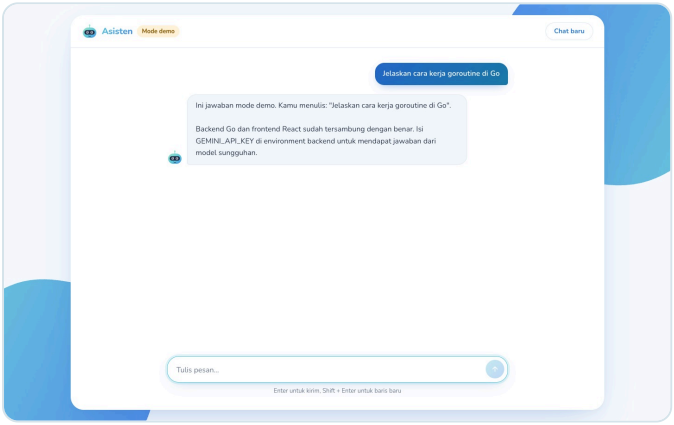
1 Clone repo & jalankan backend

Clone github.com/raihanevanza/chat-bot-go, masuk ke folder backend, lalu jalankan "go run .". Tanpa file .env, log menampilkan "provider: mock" dan server berjalan di http://localhost:8080 dalam mode demo.



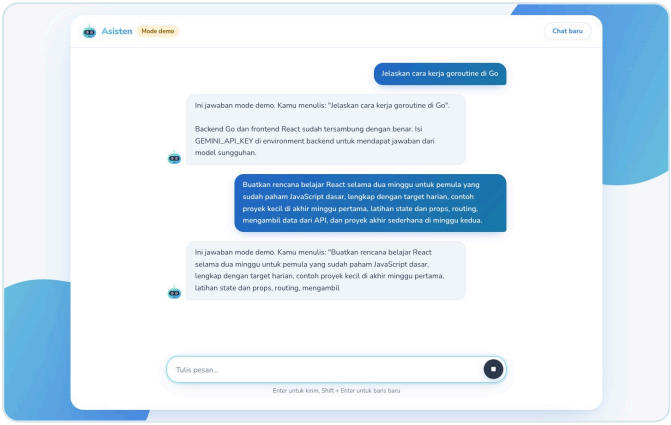
2 Jalankan frontend

Di terminal kedua, masuk ke folder frontend lalu jalankan "npm install" dan "npm run dev". Buka http://localhost:5173; badge di header menampilkan "Mode demo".



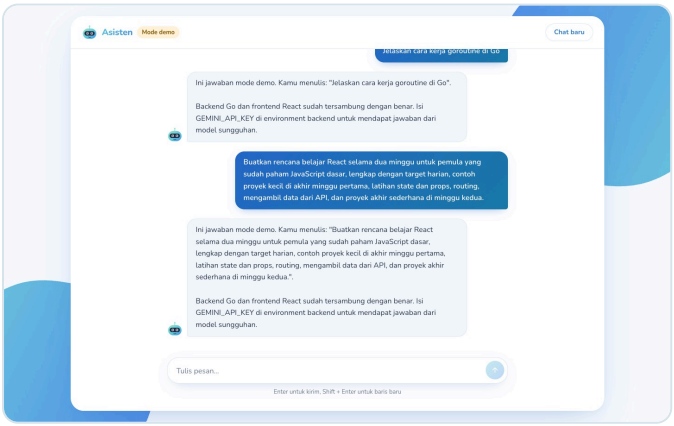
3 Kirim pesan pertama

Layar awal menyapa "Halo!" dan menampilkan tiga contoh pertanyaan. Klik salah satunya, atau ketik di kolom "Tulis pesan..." lalu tekan Enter atau tombol panah.



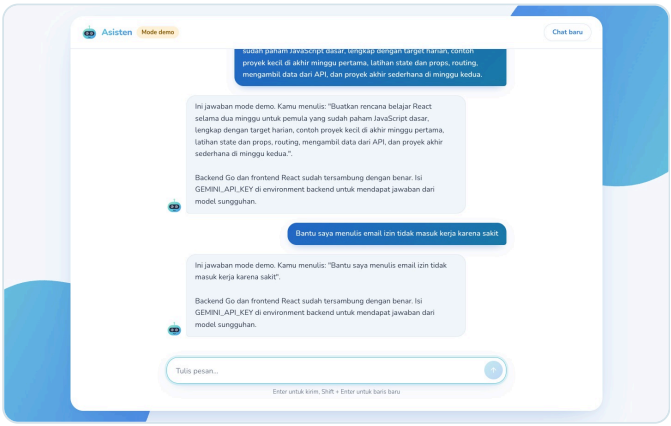
4 Lihat jawaban mengalir

Titik-titik mengetik muncul sebentar, lalu jawaban tampil kata per kata di samping maskot, dan daftar pesan bergulir otomatis mengikuti jawaban.



5 Hentikan jawaban

Selama jawaban berjalan, tombol kirim berubah menjadi tombol kotak (stop). Klik tombol itu: jawaban berhenti saat itu juga dan bagian yang sudah tampil tetap ada.



6 Lanjutkan atau mulai chat baru

Kirim pesan lanjutan untuk melanjutkan percakapan (Shift + Enter untuk baris baru). Klik "Chat baru" di kanan atas untuk mengosongkan percakapan dan kembali ke layar awal.

7 **Opsional: pakai model sungguhan**

Salin backend/.env.example menjadi .env, isi GEMINI_API_KEY (atau key OpenAI/Anthropic), lalu jalankan ulang backend. Setelah halaman dimuat ulang, badge berubah menjadi "Gemini aktif" atau "Terhubung".

KETERBATASAN & CATATAN

- Tidak ada demo live; aplikasi perlu dijalankan lokal karena memanggil API berbayar dengan key milik saya.
- Riwayat chat hanya ada di memori browser, jadi hilang saat halaman dimuat ulang. Belum ada login maupun pembatasan jumlah request.
- Jawaban ditampilkan sebagai teks biasa; format Markdown dari model (judul, daftar, blok kode) belum dirender.
- Mode demo selalu membalas dengan templat yang sama (mengulang pesan pengguna), bukan jawaban yang benar-benar menjawab pertanyaan.
- Teks antarmuka hanya berbahasa Indonesia, dan pesan error khusus baru dibuat untuk Gemini dan OpenAI. Unit test baru mencakup provider Gemini.

RENCANA BERIKUTNYA

- Merender Markdown di jawaban (misalnya dengan react-markdown) agar daftar dan blok kode tampil rapi.
- Menyimpan riwayat chat ke database (SQLite atau PostgreSQL) dan menambahkan login serta rate limiting.
- Menambah unit test untuk provider OpenAI dan Anthropic, lalu men-deploy versi demo publik.